🧪Hands-On Lab Exercise: Arithmetic Operations in Shell Script

Topic: Performing Arithmetic Operations Using Shell Script
Author: Dr. Sandeep Kumar Sharma

This lab explains how **mathematical calculations** are performed in shell scripting. Arithmetic operations allow scripts to work with numbers and perform calculations.

This is an important step in automation where scripts start doing **logical and numeric processing**.

📘What are Arithmetic Operations?

Arithmetic operations are basic math calculations: - Addition - Subtraction - Multiplication - Division - Modulus (remainder)

In simple words:

> Arithmetic operations allow shell scripts to work with numbers.

🕯️ Why Do We Need Arithmetic in Shell Script?

We need arithmetic operations to: - Perform calculations - Process numeric data - Create logical automation - Build calculators - Analyze system values - Work with counters - Build real automation logic

---

🕐Purpose of Arithmetic Operations

Arithmetic operations help scripts to: - Make decisions - Process system data - Perform monitoring - Build automation logic - Create interactive scripts - Build tools

---

📊Learning Objective

- Understand arithmetic operations in shell
- Learn numeric calculation syntax

- Perform basic calculations
- Build arithmetic scripts
- Create a calculator script

🔗Learning Outcome

After this lab, participants will: - Perform math operations in shell - Use arithmetic syntax - Build calculation scripts - Understand numeric automation - Create a calculator script

---

📌Arithmetic Syntax in Shell Script

Shell uses this syntax for calculations:

```
$(( expression ))
```

Example:

```
result=$(( 10 + 5 ))
```

🧪Example 1: Addition Script

Step 1: Create Script

```
touch add.sh
```

Step 2: Open File

```
nano add.sh
```

Step 3: Write Script

```
#!/bin/bash

a=10
b=5
sum=$(( a + b ))

echo "Addition Result: $sum"
```

Step 4: Permission

```
chmod +x add.sh
```

Step 5: Run

```
./add.sh
```

🧪 Example 2: Subtraction Script

```
#!/bin/bash

a=20
b=8
diff=$(( a - b ))

echo "Subtraction Result: $diff"
```

🧪 Example 3: Multiplication Script

```
#!/bin/bash

a=4
```

```
b=6
mul=$(( a * b ))

echo "Multiplication Result: $mul"
```

🧪 Example 4: Division Script

```
#!/bin/bash

a=40
b=5
div=$(( a / b ))

echo "Division Result: $div"
```

🧪 Example 5: Modulus Script

```
#!/bin/bash

a=17
b=3
mod=$(( a % b ))

echo "Modulus Result: $mod"
```

Understanding Arithmetic Flow

- Numbers stored in variables
- Calculation done using `$(( ))`
- Result stored in another variable
- Output printed using `echo`

🧮Final Lab Project: Shell Script Calculator

🕐Purpose:

Create a simple calculator using shell script.

Step 1: Create Script

```
touch calculator.sh
```

Step 2: Open File

```
nano calculator.sh
```

Step 3: Write Calculator Script

```
#!/bin/bash

echo "Simple Shell Calculator"

echo "Enter first number:"
read num1

echo "Enter second number:"
read num2

echo "Choose operation:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "5. Modulus"

read choice

if [ $choice -eq 1 ]
then
  result=$(( num1 + num2 ))
```

```
  echo "Result: $result"
fi

if [ $choice -eq 2 ]
then
  result=$(( num1 - num2 ))
  echo "Result: $result"
fi

if [ $choice -eq 3 ]
then
  result=$(( num1 * num2 ))
  echo "Result: $result"
fi

if [ $choice -eq 4 ]
then
  result=$(( num1 / num2 ))
  echo "Result: $result"
fi

if [ $choice -eq 5 ]
then
  result=$(( num1 % num2 ))
  echo "Result: $result"
fi
```

Step 4: Permission

```
chmod +x calculator.sh
```

Step 5: Run

```
./calculator.sh
```

📖Explanation of Calculator

- Takes two numbers as input
- Takes user choice

- Performs selected arithmetic
- Displays result
- Works like a real calculator

Key Learning

- Shell scripts can do math
- Shell scripts can process logic
- Shell scripts can take input
- Shell scripts can make tools
- Automation can include calculations

📌Summary

- Arithmetic is core to automation
- `$(( ))` is used for math
- Scripts can act as calculators
- Numeric logic builds real systems

💴End of Lab

This lab shows that shell scripts are not just commands — They can **think, calculate, and process logic**.

This is the bridge between scripting and programming.

Author:
Dr. Sandeep Kumar Sharma

🔗**LinkedIn:**
linkedin.com/in/sandeep-kaushik-2a345856

Medium Blog:
medium.com/@shyamsandeep28